Speech Processing Assignment

Name: Paruchuri Sai **RegNo: BL.EN.U4AID23063**

Dataset Description: The speech file is taken from LJ Speech Dataset from Kaggle website which contains short audio clips of speakers reading passages from books. The speech has around 9 seconds duration.

2. Deep Learning Architectures for Speech Processing

2.1 Wav2Vec2

Background

Developed by Facebook AI Research in 2020, Wav2Vec2 revolutionized automatic speech recognition (ASR) by enabling models to learn directly from raw audio—without needing transcription during pretraining. It builds on previous Wav2Vec versions and excels even with limited labeled data.

Objective

To create rich speech representations from raw audio using self-supervised learning, reducing the need for large labeled datasets.

Architecture

1. **Feature Encoder (CNN-based):** Converts raw audio into latent vectors by using convolutional layers.
2. **Transformer Encoder:** Processes these features to capture long-range dependencies.
3. **Quantization + Contrastive Loss:** Maps features into discrete representations and trains via contrastive learning.

4. **Fine-tuning:** Quantization is removed, and a linear layer is added to perform downstream tasks using loss functions like CTC.

2.2 Transformer

Background

Introduced in the landmark 2017 paper *"Attention Is All You Need"*, Transformers replaced RNNs in sequence modeling. They offer parallel processing and eliminate issues like vanishing gradients.

Objective

To model sequential data efficiently, capturing relationships across entire sequences—ideal for speech recognition and understanding.

Architecture Highlights

1. **Multi-Head Attention:** Simultaneously focuses on different parts of the input.
2. **Positional Encoding:** Injects sequence order information.
3. **Feedforward Layers:** Applies nonlinear transformation post-attention.
4. **Residual Connections & Layer Normalization:** Helps with training deep networks.
5. **Decoder (if used):** Generates output sequence using encoder outputs.

2.3 Conformer

Background

Developed by Google in 2020, the Conformer combines CNNs and Transformers to effectively capture both short-term and long-range patterns in speech.

Objective

To leverage the strength of CNNs in capturing local acoustic features and Transformers in modeling global context—ideal for phoneme-to-sentence-level recognition.

Key Components

1. **Macaron-Style Feedforward Layers (2):** Enhances feature expressiveness.
2. **Multi-Head Self-Attention:** Captures long-range speech dependencies.
3. **Convolution Module:** Encodes local information such as pitch and tone.
4. **Layer Normalization & Residuals:** Ensures training stability.
5. **Decoder:** Uses CTC or Transducer for output generation.

2.4 RNN-Transducer (RNN-T)

Background

Introduced by Alex Graves in 2012, RNN-T was designed to overcome the alignment limitations of CTC. It's now used in real-time applications by tech giants like Google and Amazon.

Objective

To enable streaming speech recognition where transcription is generated as the audio is being received.

Core Components

1. **Encoder (Acoustic Model):** Converts audio into high-level features using LSTMs or CNNs.
2. **Prediction Network:** Predicts next token using prior outputs (like a language model).
3. **Joint Network:** Merges encoder and prediction outputs to compute final probabilities.
4. **Decoding Strategy:** Allows real-time inference through methods like beam search or greedy decoding.

2.5 Connectionist Temporal Classification (CTC)

Background

Proposed in 2006 by Alex Graves, CTC allowed sequence-to-sequence training without pre-aligned data and was foundational in early end-to-end speech recognition systems.

Objective

To train models where input-output alignment is unknown—especially useful when input and output lengths vary significantly.

Working Principle

- **Encoder:** Produces per-frame predictions using RNNs or CNNs.
- **CTC Loss:** Calculates the probability over all valid alignments of the input-output pair.
- **Decoding:** Uses algorithms like beam search to derive the final transcription from output probabilities.

CTC is often used in conjunction with models like Wav2Vec2 or Conformer for efficient fine-tuning.

Comparison of outputs:

1. Transformer + CTC (Wav2Vec2) –

The transcription of the Wav2Vec is: -

IN FOURTEEN SIXTY FIVE SWAYNHEIM AND PANARCHS BEGAN PRINTING IN THE MONASTERY OF SUBIACO NEAR ROME

2. Conformer –

The transcription of the Conformer is: -

IN FOURTEEN SIXTY FIVE SWAYNHEIM AND PANARCHS BEGAN PRINTING IN THE MONASTERY OF SUBIACO NEAR ROME

3. RNN-T –

The transcription of the Conformer is: -

and fourteen sixty five swayingheim and panarax began printing in the monastery of supiako near

Inference from the Models

We evaluated the performance of three different speech recognition models—Wav2Vec2 (Transformer + CTC), Conformer, and RNN-T—by using the same sample audio file as input. Each model processed the audio independently and generated its own transcription. We then compared these transcriptions side by side and calculated the **Word Error Rate (WER)** to objectively assess their accuracy. This method helps determine how well each model interprets spoken language and where they may struggle, especially with proper nouns or complex sentence structures.

Conclusion

All three models successfully captured the core message of the audio. However, **RNN-T (Emformer RNN-Transducer)** stood out for its superior accuracy and more natural-sounding transcription.

- **Wav2Vec2 and Conformer** (both using CTC loss) delivered nearly identical outputs but showed minor inaccuracies, particularly with names like *"Sweynheim"* and *"Pannartz."*
- **RNN-T** was more precise in reproducing proper nouns and complex terms, suggesting better generalization and phoneme handling.
- In terms of fluency and correctness, **RNN-T came closest to the actual spoken sentence**, making it the best-performing model in this comparison.

GitHub code Link:
https://github.com/SaiParuchuri123/Speech_Processing